

MIGA E&S Portal 2.0

Complete Backend API & Architecture Documentation

Framework:	ASP.NET Core (C#) · MediatR CQRS
Database:	PostgreSQL (Npgsql EF Core)
Architecture:	CQRS with MediatR · Repository Pattern · DI
Auth:	WBG PolicyAuthorize · JWT Bearer
API Docs:	Swagger UI — /api/esrd/swagger
Controllers:	21 Controllers · 100+ Endpoints
Handlers:	90+ MediatR Handlers
Services:	50+ Business Services
Event Bus:	Domain Events — Azure Service Bus

1. Introduction & Backend Architecture Philosophy

The MIGA E&S Portal 2.0 backend is a carefully engineered ASP.NET Core REST API that follows the CQRS (Command Query Responsibility Segregation) pattern through the MediatR library. Every single HTTP request entering the system travels through a well-defined, predictable pipeline: it is received by a thin Controller, dispatched as a typed message to MediatR, routed to a dedicated Handler, processed by a Service, and ultimately persisted or read by a Repository that talks to the PostgreSQL database. This document traces every API endpoint through this exact journey.

The system is built as a multi-layered application. The API project itself (MIGAPortal.ESReviewDocumentation.API) contains controllers, handlers, services, validators, mappers, and interface definitions. The Domain project defines entity models and repository interfaces. The Infrastructure project implements repositories using Entity Framework Core with the Npgsql provider for PostgreSQL. A separate reusable library (MIGAPortal.RL.*) provides cross-cutting concerns like authentication, authorization, email notifications, WB Docs integration, and shared utilities.

1.1 The Golden Request Path

Every request — read or write — follows this path without exception:

HTTP Request	JWT Auth	Controller	FluentValidation	MediatR Bus	Handler	Service Layer	Repository	PostgreSQL DB	Response
Step	Layer	Responsibility						Technology	
1	HTTP Request	Browser/client sends JSON body (POST) or query params (GET) to an endpoint.						HTTP/HTTPS	REST/JSON
2	JWT Auth Middleware	ASP.NET Core authentication middleware validates the JWT Bearer token. Unauthorized requests are rejected with HTTP 401.						ASP.NET Core Authentication	JWT Bearer Token
3	Controller	The matching ApiController action receives the deserialized request model. It performs basic MVC logic — its only job is to dispatch the request.						ASP.NET Core MVC	ApiController
4	FluentValidation	Before the handler executes, MediatR's pipeline behavior automatically runs the RequestValidator FluentValidation Pipeline for the request.						FluentValidation	RequestValidator
5	MediatR Bus	MediatR resolves the correct IRequestHandler<TRequest, TResponse> for the request type using the ASP.NET Core DI container.						MediatR	IMediator
6	Handler	The handler is a single-purpose class. For Queries it calls the service and wraps the response in a CQRSHandlerResponse envelope. For Commands it calls the service and wraps the response in a CQRSHandlerResponse envelope.						CQRSHandlerResponse	CQRSHandlerResponse
7	Service Layer	All business logic lives here. Services validate business rules, orchestrate multiple repository calls, and call external integration endpoints.						Service Layer	Service Classes
8	Repository	Repositories abstract all database access. They use Entity Framework Core LINQ queries mapped to parameterized PostgreSQL queries.						Entity Framework Core	Repository Classes
9	PostgreSQL DB	Data is stored in a PostgreSQL database accessed via Npgsql. EF Core manages the database schema and migrations.						PostgreSQL	Npgsql, EF Core
10	Response	The handler returns a typed response object with STATUS='Success'/'Error', STATUS_CODE (e.g., 200 for success), NOTIFICATION (if applicable).						System.Text.Json	Response Object

1.2 Startup.cs — How Everything Is Wired Together

The Startup.cs file is the bootstrap brain of the application. Understanding it is essential because it defines how every dependency is resolved at runtime. Here is what each major registration block does:

- **services.AddAutoMapper(typeof(Startup)):**
Scans the entire assembly for all AutoMapper Profile classes and registers them. This means every MapperProfile (30+) — like MonitoringMapperProfile, DueDiligenceMapperProfile, PortfolioProjectsTabMapperProfile — is automatically available for injection into services.
- **services.AddValidatorsFromAssembly(...):**
Scans for all FluentValidation AbstractValidator classes and registers them with the DI container. These are automatically invoked by the MediatR validation pipeline behavior before any handler executes.

- **services.AddMediatR(...):**

Registers all IRequestHandler classes. MediatR will automatically discover and wire up all 90+ handlers.

- **services.AddDbContext(...):**

Registers the EF Core DbContext using the PostgreSQL connection string fetched from Azure Key Vault via the Secrets class. The Npgsql provider is configured with a custom migrations history table in the application schema.

- **services.AddHttpClient(...) + Polly RetryAsync(3):**

Registers the Malena AI service HTTP client with a 10-minute timeout and an automatic retry policy (3 retries on transient HTTP errors) using the Polly resilience library.

- **services.AddHttpClient(...):**

Registers the HTTP client for the MPM Integration Service — the external WBG system that provides pipeline/portfolio project data, contract details, and ES phase lookups. Configured with a 10-minute timeout.

- **services.InitializeAuthentication() / InitializeAuthorizationHandler():**

Configures the WBG shared authentication middleware (JWT Bearer) and the custom PolicyAuthorize attribute handler that checks ESRD-specific permissions.

- **services.InitializeDomainEventsConnection/Publisher/Receiver():**

Registers the Azure Service Bus domain event infrastructure. Events like ProjectPhaseUpdateEvent and EsInsightsMalenaPostEvent are published and received asynchronously.

- **masterDataRepository.InitializeMasterDataCache():**

Called on app startup in Configure(). Eagerly loads all master data (regions, sectors, ESRR scores, IFC tracks, project phases) into an in-memory cache so every subsequent API call can resolve lookup values without hitting the database.

2. ProjectsController — The Core Data Hub

ProjectsController is the most important controller in the system. It handles the fundamental project lifecycle operations — fetching pipeline and portfolio project lists, loading project detail, managing GBV assessments, managing team assignments, and report downloads. The controller lives at route `/api/esrd/Projects` and requires the `EsrdPermissions.ReadAll` or `EditAll` policy on every endpoint.

2.1 POST /Projects/GetPipelineProjects

Purpose: Fetch the complete list of pipeline projects (unapproved, under evaluation). This is the data source for the 'Projects in Pipeline' module grid.

Permission: EsrdPermissions.ReadAll

Request Journey — Step by Step:

■ Controller:

`ProjectsController.GetPipelineProjects([FromBody] PipelineProjectsQuery)`. The `PipelineProjectsQuery` carries optional filters: `FiscalYear[]`, `ManagementGroupCode[]`, `ProductType[]`, `EsSpecialist[]`, `ProjectCategory[]`, `HostCountryKey[]`, `RegionKey[]`, `BusinessStage[]`. Controller calls `_mediator.Send(projectsQuery)` — nothing else.

■ Handler:

`PipelineProjectsQueryHandler.Handle()` — receives the query, calls `_projectService.GetPipelineProjects(request)`, wraps the result in `PipelineProjectsResponse{Result, STATUS_CODE=0, STATUS='Success'}`.

■ Service — ProjectService.GetPipelineProjects():

This is where the heavy lifting happens. The service first calls `_commonService.GetEsMasterData(ProjectPhase)` to look up the integer code for 'Pipeline' from the master data cache. It then builds an EF Core predicate using `PredicateBuilder.True()` and adds: `esPredicate.And(x => x.ProjectPhase == pipelineValue)`. Dynamic filters are applied for `EsSpecialist` UPI list, `ProjectCategory` list. The predicate is passed to `_projectsRepository.GetPipelineInformation(esPredicate, true)`.

■ Repository — GetPipelineInformation():

Executes an EF Core query against `EsHeader` joined with `EsTeamInformations`, `EsReviewRecords` (AMR/MSV), `HighRiskProjectInfo`, `RiskHorizon`, `GbvHeaderForPipeline`, `GbvClearanceData`. Returns a list of `PipelineProjectsEntityModel` objects.

■ Integration Service Call:

The service collects all `ProjectNumbers` from the DB result and calls `_commonService.GetPipelineProjectsFromIntegrationService(projectNumbers, requiredData, functionName)`. This makes an HTTP POST to the MPM Integration Service URL (`env var MPMIntegrationServiceUrl`) to fetch: application details, milestone dates, contract data, management group, region, host country, product type.

■ Advanced Filters:

After integration data is returned, `ApplyAdvancedFilterToPipelineProjects()` applies additional filters (`FiscalYear`, `ManagementGroupCode`, `ProductType`, `RegionKey`, `HostCountryKey`, `BusinessStage`) as LINQ predicates on the in-memory list.

■ Mapping — MapPipelineProjectInformation():

For each project, the service uses `AutoMapper` to merge: `PipelineProjectsEntityModel` (DB data — ESRR scores, AMR/MSV records, team UPIs, high risk flags) with `ProjectAndApplicationDetails` (integration service data — name, country, sector, contract). User details (full names) are resolved via `_commonService.GetUserDetails(upi)`.

■ Response:

Returns List → wrapped in PipelineProjectsResponse → serialized to JSON with PascalCase property names.

Request / Response Shape:

```
// Request (POST body)
{
  "FiscalYear": [2024, 2025],           // optional – filter by MIGA FY
  "ManagementGroupCode": ["MAS"],       // optional – filter by mgmt group
  "ProductType": ["PRI"],               // optional – PRI, NH, TF
  "EsSpecialist": [123456],             // optional – lead specialist UPI
  "ProjectCategory": ["A", "B"],        // optional
  "HostCountryKey": ["ZA"],             // optional
  "RegionKey": ["AFR"]                 // optional
}

// Response envelope
{
  "STATUS": "Success",
  "STATUS_CODE": 0,
  "NOTIFICATIONS": [],
  "Result": [
    {
      "EsId": 123,
      "ProjectNumber": "14383",
      "ProjectName": "ABSA/Barclays Botswana",
      "Region": "Sub-Saharan Africa",
      "HostCountry": "Botswana",
      "Sector": "Financial",
      "FiscalYear": 2024,
      "EsDdStage": "ESDD Questionnaire",
      "ESRRDScoreText": "Partly Unsatisfactory",
      "GbvRiskRating": "Medium FI",
      "ESSpecialistFullName": "Smith, John",
      "IsHighRisk": false,
      "PipelineStatus": "PIPELINE ACTIVE"
      // ... 40+ more fields
    }
  ]
}
```

2.2 POST /Projects/GetPortfolioProjects

Purpose: Fetch all active portfolio projects (MIGA-guaranteed, operational). Powers the 'Projects in Portfolio' grid. This is the most data-intensive endpoint — it merges PostgreSQL data with Integration Service data plus cost code lookups in parallel.

Permission: *EsrdPermissions.ReadAll* | **Handler:** *PortfolioProjectsQueryHandler* | **Service:** *PortfolioProjectsService*

■ Controller:

Receives *PortfolioProjectsQuery* from POST body. Query filters: *FiscalYear[]*, *ManagementGroupCode[]*, *ProductType[]*, *ESMFiscalYear[]*, *EsSpecialist[]*, *ProjectCategory[]*, *RegionKey[]*, *HostCountryKey[]*, *BusinessStage[]*. Dispatches via *_mediator.Send()*.

■ Handler — PortfolioProjectsQueryHandler:

Single-purpose handler. Calls *_portfolioProjectsService.GetPortfolioProjects(request)* and wraps in *PortfolioProjectsResponse*.

■ Service — Step 1: DB Portfolio Records:

Builds EF Core predicate: `ProjectPhase == portfolioValue` AND optional `EsSpecialist/ProjectCategory` filters. Calls `_projectsRepository.GetPortfolioInformation(esPredicate, true)` — fetches `EsHeader` rows joined with `EsTeamInformations`, `EsReviewRecords` (AMR history + MSV data), `HighRiskProjectInfo`, `RiskHorizon`, `GbvHeaderForPortfolio`, `GbvClearanceData`.

■ Service — Step 2: Integration Service Unassigned:

Calls `_pipelineMigaPortalIntegrationDataRepository.GetPortfolioProjects()` — fetches ALL portfolio project numbers from the MPM Integration Service. Compares with DB project numbers to find 'unassigned' projects (in integration service but not yet in our DB) and adds placeholder `EsHeader` entries.

■ Service — Step 3: Parallel Calls:

Uses `Task.WhenAll()` for TWO concurrent async calls: (a) `GetProjectAndApplicationDetailsFromIntegrationService()` — fetches full application/contract/milestone data for all project numbers from MPM Integration Service; (b) `_commonService.GetProjectsCostCodesFromIntegrationService()` — fetches monitoring cost codes for all projects. This parallel execution significantly reduces total latency.

■ Service — Step 4: Apply Filters:

`ApplyAdvanceFilterToPortfolioProjects()` filters the integration service result by `FiscalYear`, `ESMFiscalYear` (ESM milestone fiscal year), `ManagementGroupCode`, `ProductType`, `RegionKey`, `HostCountryKey`, `BusinessStage` using compiled LINQ predicates.

■ Service — Step 5: Data Merge — `MapPortfolioProjectInformationResult()`:

The most complex step. For each `EsHeader` DB record: - Finds matching `ProjectAndApplicationDetails` from integration service by `ProjectNumber` - Calls `GetProjectInPortfolio()` → `_commonService.GetLegitPipelineProjectByStatus()` to get the canonical application/contract - Calls `MapProjectToPortfolioInformation()` which runs TWO AutoMapper mappings: `Map(portfolioEntityModel, projectInfo)` for DB fields then `Map(projectInPortfolio, projectInfo)` for integration fields - Sets `ContractData`: filters to active (non-terminated, non-expired) contracts, ordered by `ExpiryDate` desc - Resolves `ESSpecialistFullName`, `SupportSpecialistList`, `ManagerFullName`, `ClimateSpecialistFullName` via `GetUserDetails(UPI)` async calls - Maps AMR records: picks most recent AMR review record, checks for multiple AMRs - Maps MSV data: builds `MsvData` list for all fiscal year MSV records; identifies current FY MSV vs previous FY - Maps ESRR scores from master data cache - Calculates `ExpiringInMonths`: $((\text{expiryYear} - \text{now.Year}) * 12 + (\text{expiryMonth} - \text{now.Month}))$ - Maps GBV risk rating: prefers portfolio-phase GBV record, falls back to pipeline-phase - Sets default GBV for FI category projects if no rating found

■ Response:

Returns List (200+ projects) with full enriched data.

2.3 GET /Projects/GetProjectDetails

Purpose: Load the complete detail view for a single project — all tabs' worth of data in one call. Called when user clicks any project row in Pipeline or Portfolio.

Query params: `EsId` (int) — the `EsHeader` primary key, `IsProjectsInPipeline` (bool), `IsShelvedProject` (bool). Handler: `PortfolioDetailsQueryHandler` → `ProjectDetailsService.GetProjectDetails()`.

The service fetches the `EsHeader` record, then calls the MPM Integration Service for full project/application/contract details. It applies `GetLegitPipelineProjectByStatus()` to determine the canonical application node based on pipeline/portfolio status. It resolves all team member full names from the WBG user service. It fetches AMR history, ESAP action flags, ESRR scores, GBV risk rating, nature finance data, and document URLs. The response is a deeply nested `ProjectDetailsResponse` containing `ProjectInfo`, `ContractInfo`, `TeamInfo`, `EsrrInfo`, `GbvInfo`, and `MonitoringStatus` sections.

2.4 POST /Projects/GetGbvDetails

Purpose: Load the full GBV Risk Assessment questionnaire and scoring result for a project. Called when user opens the GBV tab.

Body: { ProjectId, EsId, HostCountryKey, IsProjectInPipeline }. Handler: GbvDetailsQueryHandler → GbvDetailsService. The service fetches GBV questionnaire answers, GBV risk rating, GBV risk score, action plan items, and GBV clearance status. The questionnaire structure (questions and options) is fetched from GBV master tables. Saved answers are merged with the question structure to populate IsAnswered flags and selected option values.

2.5 POST /Projects/SaveGbvQuestionnaires

Purpose: Persist GBV questionnaire answers and trigger GBV risk score recalculation.

Permission: EditAll. Body: { EsId, GbvId, ProjectNumber, GbvRiskRatingStatusCode, Answers[] }. Handler: SaveGbvQuestionnairesCommandHandler → GbvProjectService. Validates via SaveGbvQuestionnairesCommandValidator (all required questions must be answered). Service persists answer records to GbvQuestionsResponse table, recalculates GbvRiskScore by summing weighted option values, maps score to risk rating (Low/Medium/High/Medium FI), updates GbvHeader record.

2.6 POST /Projects/SaveTeamInformation

Purpose: Assign or update the E&S Specialist team for a project.

Body: { EsId, ProjectNumber, EsSpecialistUpi (lead), SupportSpecialistUpiList[], ProjectManagerUpi, ClimateSpecialistUpi, ClimateSupportSpecialistUpi }. Handler: SaveTeamInformationCommandHandler → TeamInfoService. Validates via SaveTeamInformationCommandValidator. Service upserts the EsTeamInformation record linked to EsId. Also calls the MPM Integration Service to update the team assignment in the external system via ClimateTeamInfoUpdatedInEsrdEvent domain event publication.

2.7 POST /Projects/CreateTeamAssignments

Purpose: Formally assign an E&S Specialist to an unassigned pipeline project, moving it from the 'Unassigned Projects' tab to 'My Projects'.

Body: { IsPipelineProject, ProjectNumber, IsESRD, ESRDComments }. Handler: SaveTeamAssignmentCommandHandler. Service creates an EsHeader record if one doesn't exist for the ProjectNumber, sets the lead specialist UPI from the authenticated user's token, sets ProjectPhase=Pipeline, and saves EsTeamInformation.

2.8 POST /Projects/SaveAdditionalInformationForPipeline / SaveAdditionalInformationForPortfolio

Purpose: Save editable metadata fields on the Project Info tab — fields like KnowledgeGap, ProjectCategory, SIPNonSIP, ESAP required flag, nature finance data, GBV status.

Both endpoints use their respective `SaveAdditionalInformation*CommandHandler` → `ProjectService`. Validated by `SaveAdditionalInformationFor*CommandValidator`. Service fetches the `EsHeader` by `EsId`, applies `AutoMapper` using `SaveAdditionalInformationForPipelineEsHeaderMapperProfile` or `SaveAdditionalInformationForPortfolioEsHeaderMapperProfile`, then updates and saves.

3. DueDiligenceController — ESRR Assessment Engine

The Due Diligence API drives the most critical E&S evaluation workflow. An E&S Specialist uses these endpoints to score a project against IFC Performance Standards (PS1–PS8) through a structured questionnaire, generating the ESRR(D) risk rating that determines whether a project receives its MIGA guarantee. Route: `/api/esrd/DueDiligence`.

3.1 GET /DueDiligence/GetDueDiligenceDetails

Purpose: Load the full Due Diligence questionnaire for a project — all sections, all questions, previously saved answers, and the computed ESRR(D) rating.

Query param: EsId (int). Handler: GetDueDiligenceDetailsQueryHandler → DueDiligenceService.GetDueDiligenceDetail().

■ EsHeader Fetch:

Calls `_iDueDiligenceRepository.GetByIdAsync(esId)` — fetches the root project header which contains `IsEsap`, `EsStage`, `EsrrdScore`, `EsrrdDate`, `IsEsrrdNa`.

■ Contract Editability Check:

Calls `_projectDetailsService.getProjectDetailsFromReusableService(projectNumber)` to fetch contract data from the MPM Integration Service. Then calls `_commonService.IsDueDiligenceEditable(contractEffectiveDate)` — returns true if the contract effective date is within an editable window (used to enable/disable the Save button in the UI).

■ Question Masters:

Fetches `PsQuestionMaster` records (all PS/ESRR/PSAR questions with their types, sections, sequence numbers) and `QuestionOptionsMaster` records (option choices per question) from the DB.

■ Saved Answers:

Calls `_iDueDiligenceRepository.GetDueDiligenceResponses(esId)` — fetches `DdQuestionResponseSummary` (the summary record containing `EsrrRating`, `EsrrOverridenRating`, `EsrrAccessDate`, `GeneralComments`) and all `DdQuestionsResponse` child records (one per question answered).

■ Questionnaire Assembly — MapPssdQuestions():

`_questionnaireService.MapPssdQuestions()` merges PS question masters with saved answers. For each PS question, it looks up the corresponding `DdQuestionsResponse` by (QuestionSection, QuestionId) and marks `IsAnswered=true`, populates the selected `OptionId`.

■ Questionnaire Assembly — MapEsrrQuestions():

`MapEsrrQuestions()` builds the ESRR scoring questionnaire (Management, Performance, Communication factor groups) with saved scores populated.

■ Questionnaire Assembly — MapPsarQuestions():

`MapPsarQuestions()` builds the PS Applicability & Rating section. For each PS (1-8), shows whether it applies and the specialist's assessment.

■ Contextual Risk Questions:

`MapContextualRiskQuestions()` fetches `ContextualRiskBaseQuestion`, `ContextualRiskMappingQuestion`, `ContextualRiskSubMappingQuestion` from DB. Builds a 3-level nested question hierarchy. Populates saved answers from `DdQuestionsResponse` where `QuestionSection = 'CR', 'CRM', 'CRSMQ'`.

■ ESRR Score Text Resolution:

If `EsrrRatingCode` is a non-zero integer, calls `_commonService.GetEsrrScoreText(code)` to resolve it to human-readable text ('Satisfactory', 'Partly Unsatisfactory', etc.) from the master data cache.

■ ES Input Info:

`MapEsInputInfo()` fetches PS question masters for the `EsInput` section and returns them for the 'Is ESAP Required?' question.

■ Bubble Information:

`MapBubbleInformation()` fetches hover tooltip data for question fields.

■ Response:

Returns `DueDiligenceQuestionnaire` with: `ProjectStageId`, `IsDueDiligenceEditable`, `ResponseSummary` (ratings, scores, comments), `PssdQuestionnaires[]`, `EsrrQuestionnaires[]`, `PsarQuestionnaires[]`, `ContextualRiskQuestionnaires[]`, `EsInput`, `BubbleInformation`, `EsrrFactorRating`.

3.2 POST /DueDiligence/SaveDueDiligence

Purpose: Persist the complete Due Diligence questionnaire responses and update the ESRR(D) score on the `EsHeader` record.

Permission: EditAll. Body: SaveDueDiligenceCommand. Validated by SaveDueDiligenceCommandValidator (ContextualRisk section questions must have an OptionId selected). Handler: SaveDueDiligenceCommandHandler → DueDiligenceService.SaveDueDiligenceResponse().

■ Override Flag Handling:

If `GeneralInformation.DeleteEsrrOverridenRatingInfo` is true, the service sets `EsrrOverridenComments=null` and `EsrrOverridenRatingCode=null` — clearing a manager's override.

■ EsHeader Validation:

Fetches `EsHeader` by `EsId`. If the project is already in Monitoring stage and not in an editable window, an error notification is added and 0 is returned (handled by the handler as an error response).

■ First-Time Save Path — No Existing DdQuestionResponseSummary:

If no `DdQuestionResponseSummary` exists for this `EsId`: `AutoMapper` creates a new `DdQuestionResponseSummary` from `GeneralInformation` (contains `EsrrRating`, `EsrrOverridenRating`, `EsrrAccessDate`, `Comments`). Maps `QuestionDetails[]` to `DdQuestionsResponse[]` (one DB record per question). Sets `DdQuestionsResponseId=0` on each to force INSERT. Updates `EsHeader.EsrrdScore` and `EsrrdDate`. Calls `_projectsRepository.InsertAsync(newDdResponseSummary) + SaveAsync()`.

■ Update Path — Existing DdQuestionResponseSummary:

If a record already exists: `AutoMapper` updates the existing `DdQuestionResponseSummary` from `GeneralInformation`. For each `QuestionDetail`: looks up the existing `DdQuestionsResponse` by (`QuestionSection`, `QuestionId`). If found: `AutoMapper` updates it in-place. If not found: creates a new `DdQuestionsResponse` and adds to the list. Calls `_iDueDiligenceRepository.Update() + SaveAsync()`.

■ ESAP Flag Sync:

After saving, the service checks the response to question ID=11 in the `EsInput` section. If `OptionId=1` (Yes), sets `EsHeader.IsEsap=true`. This keeps the ESAP required flag in sync with the DD questionnaire answer.

■ EsStage Update:

If no monitoring record exists yet, sets `EsHeader.EsStage = 'DD'` and `EsStageId = projectStageId` (the `DdProjectStageId`). If monitoring already exists, the stage is not downgraded.

■ Return:

Returns the DdProjectStageId (non-zero) on success, or 0 on error.

3.3 POST /DueDiligence/DeleteDueDiligence

Purpose: Remove a complete Due Diligence assessment record, resetting the project back to an unassessed state.

Query param: ProjectStageId (int). Permission: EditAll. Handler: DeleteDueDiligenceQueryHandler → DueDiligenceService.DeleteDueDiligence(). Service: Fetches DdQuestionResponseSummary by ProjectStageId to get EsId. Fetches EsHeader. Calls _iDueDiligenceRepository.DeleteDueDiligence(projectStageId) which cascades deletion of DdQuestionsResponse children. Resets EsHeader: EsStageId=0, EsrrdScore=null, EsrrdDate=null. Special case: if ProjectCategory is 'NA' type, sets IsEsrrdNa=true.

3.4 POST /DueDiligence/DownloadDueDiligenceReport

Purpose: Generate and download the official E&S Due Diligence report as a Word or PDF document.

Body: DownloadDueDiligenceReportQuery (contains EsId, ReportFormat, full questionnaire section data). Permission: ReadAll. Handler: DownloadDueDiligenceReportQueryHandler → DownloadDueDiligenceQuestionnaireReportService. Service uses the Aspose library (SetAsposeLicences() called at startup) to generate a structured Word document containing: project header, PS applicability table, all factor scores with justifications, ESRR(D) rating with override info, specialist comments. Returns as DownloadFileResponse{RESULT: byte[], FILE_NAME}. Controller wraps in FileStreamResult with appropriate MediaType.

4. MonitoringController — Portfolio Monitoring Engine

The Monitoring API manages all E&S monitoring activities for portfolio projects. Once a project moves from Pipeline to Portfolio (receives its MIGA guarantee), the E&S Specialist uses these endpoints to record annual AMR reviews, site visits, virtual monitoring sessions, and client meetings. The monitoring workflow is the most business-logic-heavy area of the backend. Route: `/api/esrd/Monitoring`.

4.1 POST `/Monitoring/SaveMonitoringActivity`

Purpose: Create a new monitoring activity record (first-time creation of a monitoring period for a project). This is the main entry point for recording an AMR review, site visit, virtual monitoring, client meeting, or combination of these.

Method: [FromForm] multipart — supports optional file uploads. Permission: EditAll. Validated by SaveMonitoringActivityCommandValidator. Handler: SaveMonitoringActivityCommandHandler → MonitoringService.SaveMonitoringActivity().

■ EsHeader Validation:

Fetches EsHeader by EsId. If not found, adds error notification 'EsId not found' and returns 0.

■ Contract Validation:

Calls `_commonService.CheckIfProjectContainsContracts(projectNumber)` — makes an HTTP call to the MPM Integration Service to verify the project has at least one active contract. Without an active contract, monitoring cannot be created.

■ New Record Path (`ProjectStageId == 0`):

Creates a new EsMonitoring entity from the command via AutoMapper.

■ Questionnaire Seeding — No Existing Monitoring:

If this is the first monitoring record for the project: looks up the existing DueDiligence responses (`_iDueDiligenceRepository.GetDueDiligenceResponses()`). Maps them to MnQuestionResponseSummary via AutoMapper (`DdQuestionResponseSummary→MnQuestionResponseSummary`). Sets EsId on all child MnQuestionsResponse records. This seeds the monitoring questionnaire with the DD questionnaire answers as a starting point.

■ Questionnaire Seeding — Existing Monitoring Records:

If monitoring records exist: fetches the most recent MnQuestionResponseSummary (ordered by EsrrAccessDate desc, then FiscalYear desc). Clones it as a new summary (IDs reset to 0 for INSERT). This 'forward seeds' the questionnaire from the previous monitoring period.

■ DB Save:

Calls `_monitoringRepository.InsertAsync(monitored) + SaveAsync()`. The auto-generated ProjectStageId is captured from the inserted record.

■ AMR Receipts Update:

If `AmrReviewFlag=true` AND `AmrDueDate` has value: calls `UpdateAmrReceiptsRecord()`. This finds the EsReviewRecord where `EsId=this` AND `DueNextDate=AmrDueDate` AND `ReviewType='AMR'`. Updates: `IsAmrReviewed`, `AmrReviewedDate`, `AmrReviewStartDate`, `LastReceivedDate` (`= AmrReceivedDate`), `LastReceivedComments`, `AmrCarriedOut`.

■ Site Visit MSV Record:

If SiteVisitFlag=true AND SiteVisitToDate has value: calls CreateOrUpdateRecord(). Gets fiscal year from SiteVisitToDate. Finds EsReviewRecord for this EsId+MSV+FiscalYear. If found: sets IsMsvCompleted=true, MsvCompletionDate=SiteVisitToDate. Also calls UpdateLastMsvDateFuture() to set the LastReceivedDate on the NEXT fiscal year's MSV record.

■ AMR Required Auto-Set:

If AmrReviewFlag=true and EsHeader.IsAmrRequired was false: auto-sets IsAmrRequired=true, AmrDueDate=new due date, AmrDueDay/Month/Year, AmrDueFrequency='Yearly'.

■ EsHeader Stage Update:

Sets EsHeader.EsStage='MN', EsStageId=new ProjectStageId. If monitoring questionnaire summary exists: updates EsrmDate and EsrmScore (using override rating if set, else calculated rating). Calls _projectsRepository.UpdateSingle(esHeader) + SaveAsync().

4.2 POST /Monitoring/SaveMonitoring (Update Questionnaire)

Purpose: Update the monitoring questionnaire answers (ESRR factors, PS scores) and monitoring activity details for an existing monitoring period.

Body: SaveMonitoringCommand (EsId, ProjectStageId, MonitoringActivityInformation, GeneralInformation with ESRR scores, QuestionDetails[]). Permission: EditAll. Handler: SaveMonitoringCommandHandler → MonitoringService.SaveMonitoring().

Service validates EsHeader is not in DD stage (monitoring questionnaire only editable in MN stage). Fetches existing EsMonitoring + MnQuestionResponseSummary by (EsId, ProjectStageId) with navigation properties loaded. AutoMapper updates the monitoring entity and summary from command data. For each QuestionDetail: finds existing MnQuestionsResponse by (QuestionSection, QuestionId) — updates if found, creates new if not. Removes existing HighRisk section responses (QuestionSection = 'HR', 'HRM', 'HRSMQ') before saving new ones. After save: re-fetches the latest MnQuestionResponseSummary with EsrmAccessDate set, updates EsHeader.EsrmScore and EsrmDate. If ContractExpiryDate provided: recalculates KnowledgeGap and UpcomingKnowledgeGap using _commonService.CalculateKnowledgeGapUsingEsData().

4.3 POST /Monitoring/SaveMonitoringActivityAndAmrHistory

Purpose: Save a monitoring activity AND simultaneously record it in the AMR history table. Used when the system needs both the monitoring record and the historical AMR log in one atomic operation.

Body: SaveMonitoringActivityAndAmrHistoryCommand. Permission: EditAll. Handler: SaveMonitoringActivityAndAmrHistoryCommandHandler. Validated by SaveMonitoringActivityAndAmrHistoryCommandValidator. The handler calls MonitoringService.SaveMonitoringActivity() first, then calls AmrHistoryDetailsService.SaveAmrHistoryDetails() to record the AMR event in the history table for reporting purposes.

4.4 GET /Monitoring/GetMonitoringDetails

Purpose: Load the full monitoring questionnaire for a specific monitoring period (ProjectStageId). Called when user opens a monitoring record to view or edit.

Query params: EsId, ProjectStageId (0 = load latest). Handler: GetMonitoringDetailsQueryHandler → MonitoringService.GetMonitoringDetail(). Service: Fetches EsHeader. Fetches AMR history records for the project. If

ProjectStageId=0: determines latest by calling GetLatestMonitoringActivity() (fetches MnQuestionResponseSummary ordered by EsrrAccessDate desc, returns ProjectStageId). Fetches documents for MonitoringCode stage. Fetches PsQuestionMaster and QuestionOptionsMaster. Fetches MnQuestionResponseSummary + child MnQuestionsResponses by (EsId, ProjectStageId). Assembles MonitoringQuestionnaire object: maps MonitoringActivityInformation, maps virtual monitoring documents, calls MapResponseSummary/MapPssdQuestions/MapEsrrQuestions/MapPsarQuestions/MapEsInputInfo/MapBubbleInformation. Also calls GetMonitoringList() to populate the monitoring history tab.

4.5 GET /Monitoring/GetMonitoringList

Purpose: Fetch the list of all monitoring activity records for a project. Powers the monitoring history table in the UI.

Query param: EsId. Handler: GetMonitoringListQueryHandler → MonitoringService.GetMonitoringList(). Calls _monitoringRepository.GetMonitoringList(esId) — returns all EsMonitoring records for the project. Also fetches documents filtered to MonitoringCode stage. For each monitoring record, builds a MonitorList DTO with all date fields formatted as MM/DD/YYYY, AMR and Site Visit flags, FinalEsrr text (score code + text description), VirtualMonitoring documents list. Returns List.

4.6 GET /Monitoring/GetDueDateForProject

Purpose: Retrieve all AMR due date options for a project — used to populate the 'AMR Due Date' dropdown in the monitoring activity form.

Query param: EsId. Handler: GetDueDateQueryHandler → MonitoringService.GetDueDate(). Queries EsReviewRecord where EsId=this AND ReviewType='AMR' AND DueNextDate is not null. Returns List< of distinct due dates formatted as MM/DD/YYYY, ordered descending.

4.7 POST /Monitoring/DeleteMonitoring

Purpose: Delete a monitoring activity record and re-sync the EsHeader stage and ESRR score.

Query param: ProjectStageId. Permission: EditAll. Handler: DeleteMonitoringQueryHandler → MonitoringService.DeleteMonitoringData(). Service: Calls _monitoringRepository.DeleteMonitoringData(projectStageId) — cascades deletion of MnQuestionResponseSummary and MnQuestionsResponse. Re-queries remaining monitoring list for this EsId. If records remain: finds the latest MnQuestionResponseSummary with EsrrAccessDate set; updates EsHeader.EsStageId, EsrrmDate, EsrrmScore to the latest remaining record's values. Recalculates KnowledgeGap. If NO monitoring records remain: falls back to DueDiligence stage — fetches DdQuestionResponseSummary, sets EsHeader.EsStageId = DdProjectStageId or 0, sets EsStage='DD', clears EsrrmScore/EsrrmDate, resets KnowledgeGap flags.

5. EsapActionController — Compliance Action Plan Engine

The ESAP (Environmental and Social Action Plan) API manages the contractual compliance actions that guarantee holders must fulfill. E&S Specialists create, track, update, and verify completion of these actions throughout both the Pipeline and Portfolio phases. Route: /api/esrd/EsapAction.

5.1 GET /EsapAction/GetEsapDetails

Purpose: Load the complete ESAP section — including whether ESAP is required, all action items with history, and any uploaded ESAP documents.

Query param: EsId. Validated by GetEsapDetailsQueryValidator (EsId must be > 0). Handler: GetEsapDetailsQueryHandler → EsapActionService.GetEsapDetails().

Service builds EF Core predicate on EsHeader.Id==EsId and calls `_esapActionRepository.GetEsapDetails(predicate)` — fetches EsHeader with navigation properties: EsapActionInformations (joined with EsapActionHistories), EsDocuments (filtered to ESAP type). Maps EsapActionInformations to EsapActionInformationDto list via AutoMapper. For each action's history entries: resolves LastUpdatedBy UserDetails via `GetUserInfo(UIP)`. Filters documents to those of type ESAP with non-empty DocumentSection. Resolves document type display names via `_masterDataRepository.GetEsrdDocumentMasterData()`. Checks `IsEsapActionUploadAllow`: false if any ESAP documents with Status=Final already exist (prevents duplicate uploads). Fetches the 'Is ESAP Required?' question text from PsQuestionMaster where Section='EsInput'. Returns EsapDetailsResponseDto with EsapRequiredInfo, EsapActions[], EsapDocuments[], IsEsapActionUploadAllow.

5.2 POST /EsapAction/SaveEsapAction

Purpose: Create a new ESAP action item or update an existing one, with automatic history tracking for rescheduling.

Body: SaveEsapActionCommand. Permission: EditAll. Validated by SaveEsapActionCommandValidator (Action text and DueDate required). Handler: SaveEsapActionCommandHandler → EsapActionService.SaveEsapAction().

■ EsHeader Validation:

Fetches EsHeader by EsId. Checks `IsEsap` flag — if ESAP is not enabled for this project (`IsEsap=false`), returns error 'ESAP not required for this project'.

■ New Action (EsapActionId == 0):

AutoMapper creates EsapAction from command. Sets `ProjectStageType = EsHeader.EsStage` (current stage code). Calls `_esapActionRepository.AddEsapAction(esapAction)`. Returns newly generated EsapActionId.

■ Update Action (EsapActionId > 0):

Fetches existing EsapAction by EsapActionId. If record not found: adds error notification and returns 0. Checks if RescheduleDate has changed — if so, creates EsapActionHistory record (EsapActionId, new RescheduleDate, Remarks) to capture the rescheduling event via `_esapActionRepository.AddEsapActionHistory()`. AutoMapper updates the existing EsapAction from the command. Updates `ProjectStageType` to current stage. Calls `_esapActionRepository.UpdateEsapAction()`. Returns EsapActionId.

5.3 POST /EsapAction/DeleteEsapAction

Purpose: Delete one or multiple ESAP action items (bulk deletion via checkbox select).

Body: DeleteEsapActionCommand{ esapActionIdList: int[] }. Permission: EditAll. Validated by DeleteEsapActionCommandValidator. Handler: DeleteEsapActionCommandHandler → EsapActionService.DeleteEsapAction(list). Repository calls _esapActionRepository.DeleteEsapAction(idList) which soft-deletes records (sets IsDeleted=true) or hard-deletes depending on repository implementation.

5.4 POST /EsapAction/UploadEsapAction (Template Upload)

Purpose: Upload a filled ESAP Action Plan Excel template to bulk-import ESAP actions.

Form: [FromForm] UploadEsapActionCommand (EsId, IFormFile). Permission: EditAll. Handler: UploadEsapActionCommandHandler → EsapActionService.UploadEsapActionPlanDetails(). Service: Validates the uploaded Excel file structure against template column definitions (IsValidEsapActionTemplate()). Reads rows using a spreadsheet library (Aspose.Cells). Deletes existing non-deletable ESAP actions (_esapActionRepository.GetEsapAction where !IsDdDeletable → DeleteMultiple). Maps rows to EsapAction entities. Sets EsId and ProjectStageType on each. Calls _projectsRepository.InsertMultipleAsync() + SaveAsync().

5.5 POST/GET /EsapAction/SaveEsapRequired | GetEsapRequired

Purpose: Toggle whether an ESAP is required for a project, keeping the EsHeader flag and DD questionnaire answer in sync.

SaveEsapRequired: Body: { EsId, IsEsapRequired: bool }. Service calls _esapActionRepository.GetEsapRequiredWithDdResponse(EsId) — fetches EsHeader with related DdQuestionsResponses navigation. Sets EsHeader.IsEsap = IsEsapRequired. Finds the DD question response for QuestionId=11 (the 'Is ESAP Required?' question in EsInput section). Sets OptionId=1 (Yes) if IsEsapRequired=true, else OptionId=2 (No). Updates both records atomically. GetEsapRequired: Simply fetches EsHeader.IsEsap value.

5.6 POST /EsapAction/DownloadEsapActionReport | DownloadEsapActionTemplate

Purpose: Generate ESAP report as Excel/PDF, or download the blank ESAP template.

DownloadEsapActionReport: Body: DownloadEsapActionReportCommand (EsapActions list). Handler: DownloadEsapActionReportCommandHandler → EsapActionService.PrepareAsposeData(). Creates Workbook using Aspose.Cells, calls PrepareESAPActionWorksheet() which sets up column widths, document title, header row, and data rows. Applies date formatting to columns B/C/D. DownloadEsapActionTemplate: Reads the pre-built template file from disk (Constants.EsapApctionPlanTemplate path). Returns raw bytes. Both return FileStreamResult.

6. DashboardController — Analytics & Reporting Hub

The Dashboard API aggregates data from across the entire system to power the analytics hub. It includes the main dashboard (all projects), pipeline-specific analytics, portfolio analytics, and report download endpoints. Route: /api/esrd/Dashboard.

6.1 POST /Dashboard/GetDashboardDetails (Manager vs Specialist variants)

Purpose: Load the complete dashboard analytics — project counts, ESRR distributions, AMR status, MSV summary, business stage counts, contract expiry stats, GBV summary. Two variants: Manager (all projects) requires DashboardManager permission; Specialist variant requires DashboardSpecialistView and scopes data to the logged-in user's projects.

Body: DashboardQuery { EsSpecialistUPI? (0=all), ManagementGroupCode[], FiscalYear[], ProductType[], RegionKey[] }.

Handler: DashboardDetailsQueryHandler → DashboardService.GetDashboardDetails(). Service: Fetches current user via _commonService.GetCurrentUser() to check policy (DashboardManager vs DashboardSpecialistView). Fetches ALL EsHeader rows (pipeline + portfolio) via _projectsRepository.GetAllEsProjectsDataForDashboard() with includes for team, AMR records, MSV records, contract data. If EsSpecialistUPI > 0: filters to projects where lead UPI or support UPI list matches. Policy fallback: if specialist filter yields empty result (no own projects), defaults to all-projects view. Runs TWO parallel queries via Task.WhenAll: GetPipelineProjectsFromIntegrationService (application/contract data) and GetProjectsCostCodes (cost codes). Computes: pipeline count by status (ACTIVE/LEAD/SLOW MOVING), portfolio count, upcoming expiry count (contracts expiring in next 12 months), ESRR(D) and ESRR(M) distributions, AMR summary (projects with AMR, received this FY/CY, outstanding, overdue), MSV summary, category distribution (A/B/C/FI), management group distribution, GBV summary. Returns richly structured DashboardDetails object.

6.2 POST /Dashboard/GetPipelineProjectsForDashboardDetails / GetPortfolioProjectsForDashboardDetails

Purpose: Dedicated pipeline and portfolio analytics endpoints that power the tab-specific dashboard sections with finer-grained metrics.

Pipeline handler: PipelineProjectsDashboardQueryHandler → DashboardService pipeline-specific methods. Returns: pipeline project count, count by DD stage (Questionnaire/Clearance/Negotiation/ESRS Prepared/Disclosed), count by region, count by management group, ESRR(D) distribution. Portfolio handler: PortfolioProjectsDashboardQueryHandler → DashboardService portfolio-specific methods. Returns: portfolio count, AMR required count, AMR received FY/CY, AMR outstanding, MSV recommended/completed/not-completed counts, Track B count, upcoming expiry (6/12 month buckets), ESRR(M) distribution, knowledge gap count.

6.3 POST /Dashboard/DownloadDashboardReport (and variants)

Purpose: Generate and download the dashboard summary as an Excel/PDF report. Separate endpoints for full dashboard, pipeline-only, portfolio-only, and other initiatives.

Each download endpoint follows the same pattern: DownloadDashboard*ReportsQuery → handler → service → Aspose Excel generation → DownloadFileResponse{RESULT:byte[], FILE_NAME}. Controller wraps byte[] in

MemoryStream → FileStreamResult with MediaType header. Services create multi-worksheet Excel workbooks using Aspose.Cells, with styled headers, data rows, and column width auto-fitting.

7. ReportsController — Compliance Report Generation

The Reports API provides all pre-built compliance and analytics reports. Every report follows the same pattern: receive filter parameters, query the DB (and optionally the integration service), generate an Excel/PDF using Aspose, and stream the file. Route: /api/esrd/Reports.

Endpoint	Handler	Service	Description
POST GetReportDetails	GetReportDetailsQueryHandler	ReportDetailsService	Returns structured data for the Reports landing page.
GET GetTrendingEsrrReportDetails	GetTrendingEsrrReportDetailsQueryHandler	GetTrendingEsrrReportDetailsService	Fetches historical ESRR score data across all projects.
GET GetTrendingEsrrSummaryReportDetails	GetTrendingEsrrSummaryReportDetailsQueryHandler	GetTrendingEsrrSummaryReportDetailsService	Summary version of ESRR trends — counts per score.
GET GetAmrSubmissionReportDetails	GetAmrSubmissionReportDetailsQueryHandler	AmrSubmissionReportDetailsService	AMR submission data grouped by FY and market.
GET GetMsvReportDetails	GetMsvReportDetailsQueryHandler	MsvReportDetailsService	MSV data for a given FY: recommended, planned, and completed.
POST DownloadHighRiskReport	DownloadHighRiskReportQueryHandler	DownloadHighRiskReportService	Excel/PDF report of all current High Risk projects.
POST DownloadHighRiskListReport	DownloadHighRiskListReportQueryHandler	DownloadHighRiskListReportService	Simplified list format of High Risk projects.
POST DownloadHighRiskRemovalReport	DownloadHighRiskRemovalReportQueryHandler	DownloadHighRiskRemovalReportService	Projects removed from High Risk classification.
POST DownloadHighRiskRemovalListReport	DownloadHighRiskRemovalListReportQueryHandler	DownloadHighRiskRemovalListReportService	Simplified list format of High Risk removals.
POST DownloadEsapStatusReport	DownloadEsapStatusReportQueryHandler	DownloadEsapStatusReportService	All ESAP actions across portfolio: Open/Overdue.
POST DownloadEsrrStatusReport	DownloadEsrrStatusReportQueryHandler	DownloadEsrrStatusReportService	ESRR(M) scores for all portfolio projects — current and historical.
POST DownloadAmrSubmissionReport	DownloadAmrSubmissionReportQueryHandler	DownloadAmrSubmissionReportService	Detailed AMR submission status per project with comments.
POST DownloadMonitoringVisitTrendsReport	DownloadMonitoringVisitTrendsReportQueryHandler	DownloadMonitoringVisitTrendsReportService	Site visit trend analysis across fiscal years.
POST DownloadEsrrScoreForPortfolioReport	DownloadEsrrScoreForPortfolioReportQueryHandler	DownloadEsrrScoreForPortfolioReportService	ESRR score history for all portfolio projects.
POST DownloadEsrrScoreForIndividualProjectReport	DownloadEsrrScoreForIndividualProjectReportQueryHandler	DownloadEsrrScoreForIndividualProjectReportService	ESRR score history for a single project across fiscal years.
POST DownloadMonitoringProjectReport	DownloadMonitoringProjectReportQueryHandler	DownloadMonitoringProjectReportService	Full monitoring activity report for a project.
POST DownloadNatureFinanceReport	DownloadNatureFinanceReportQueryHandler	DownloadNatureFinanceReportService	Nature finance assessments across pipeline and projects.
POST DownloadMajorIncidentsReport	DownloadMajorIncidentsQueryHandler	DownloadMajorIncidentsReportService	E&S major incidents above severity threshold.
POST DownloadRecentlyUpdatedProjectsReport	DownloadRecentlyUpdatedProjectsReportQueryHandler	DownloadRecentlyUpdatedProjectsReportService	Record changes in last 30/60/90 days.

8. Remaining Controllers — Detailed API Reference

8.1 HighRiskController — /api/esrd/HighRisk

Manages the High Risk classification for portfolio projects. A project is classified High Risk when it has major E&S compliance failures, stakeholder complaints, or CAO involvement.

Endpoint	Method	Description
GET GetHighRiskDetails	GetHighRiskDetailsQueryHandler → HighRiskService	HighRiskService reads high risk data (IsHighRisk, HighRiskAdditionDate, HighRiskReasons) for a given EsId.
POST SaveHighRiskDetails	SaveHighRiskDetailsCommandHandler → HighRiskService	HighRiskService updates HighRiskProjectInfo record. If IsHighRisk=true and was previously false: s

8.2 AuditSummaryController — /api/esrd/AuditSummary

Provides the complete audit trail for a project. Every field change on any project entity is captured by EF Core's built-in audit logging mechanism.

GET GetAuditSummary (query param: EsId): Permission ReadAll. Handler: AuditSummaryQueryHandler → AuditSummaryService. Service queries AuditSummaryRepository which reads from the AuditLog table (or EF Core shadow property audit) filtered by EntityId=EsId. Returns AuditSummaryResponse: list of audit entries with EntityType, ChangedBy (UPI), ChangedAt, FieldName, OldValue, NewValue, Operation (Insert/Update/Delete). Used to render the 'Audit Summary' tab on project detail — read-only view of all changes.

8.3 DocumentController — /api/esrd/Document

Manages generic E&S documents for projects (non-ESAP documents, monitoring documents, GBV documents).

Endpoint	Description
POST SaveDocument [FromForm]	Handler: SaveDocumentCommandHandler → DocumentService. Receives multipart form with EsId, DocumentType, DocumentContent.
POST DeleteDocument	Handler: DeleteDocumentCommandHandler → DocumentService. Marks document as deleted in DB (IsDeleted=true).
POST UpdateDocumentNotes	Handler: UpdateDocumentNotesCommandHandler → DocumentService. Validated by UpdateDocumentNotesCommandValidator. Updates document notes.
GET GetDocuments	Handler: GetDocumentsQueryHandler → DocumentService. Fetches all EsDocuments for given EsId and optionally filtered by DocumentType.

8.4 NatureFinanceController — /api/esrd/NatureFinance

Tracks nature-based finance assessments aligned with global biodiversity frameworks.

GET GetNatureFinanceDetails (EsId): Handler: GetNatureFinanceQueryHandler → NatureFinanceService. Validated by GetNatureFinanceQueryValidator. Fetches NatureFinance entity from NatureFinanceRepository by EsId. Returns NatureFinanceDetailsResponse: IsNatureFinance (applicable), NatureFinancePotential (1-5 score), Justification text.
 POST SaveNatureFinance: Handler: SaveNatureFinanceCommandHandler → NatureFinanceService. Validated by SaveNatureFinanceCommandValidator. Upserts NatureFinance record — InsertAsync if new, Update if exists. Returns SaveNatureFinanceResponse with saved values.

8.5 OtherInitiativesController — /api/esrd/OtherInitiatives

Manages non-project E&S workstreams (WBG/IFC collaborations, CAO policy items).

Endpoint	Description
GET GetOtherInitiatives	Handler: OtherInitiativesQueryHandler → OtherInitiativesService. Fetches all OtherInitiative records from OtherInitiativesService.
POST SaveOtherInitiatives	Handler: AddOtherInitiativesCommandHandler → OtherInitiativesService. Validated by SaveOtherInitiativesCommandValidator. Saves OtherInitiatives.
POST DeleteOtherInitiatives	Handler: DeleteOtherInitiativesCommandHandler → OtherInitiativesService. Soft-deletes OtherInitiative by Id.
POST AddOtherInitiativesType	Handler: AddOtherInitiativesTypeCommandHandler. Validated by AddOtherInitiativesTypeCommandValidator. Adds a new OtherInitiativesType.
POST DownloadOtherInitiativesReport	Handler: DownloadOtherInitiativesReportQueryHandler → DownloadOtherInitiativesReportService. Generates Excel report of all other initiatives. Uses DownloadOtherInitiativesReportService with Aspose.Cells.

8.6 IncidentController — /api/esrd/Incident

Tracks major E&S incidents associated with portfolio projects.

Endpoint	Description
POST SaveIncident [FromForm]	Handler: SaveIncidentCommandHandler → IncidentService. Validated by SaveIncidentCommandValidator. [FromForm] saves incident details.
POST DeleteIncident	Handler: DeleteIncidentCommandHandler (Document folder). Soft-deletes incident and related documents.
GET GetIncidents	Handler: GetIncidentListQueryHandler → IncidentService. Returns paginated IncidentListResponse.
GET GetIncidentDetail	Handler: GetIncidentDetailQueryHandler → IncidentService. Returns full IncidentDetailResponse with description, type, and status.
POST UploadDocumentForIncident	Handler: UploadDocumentForIncidentCommandHandler. Uploads a file to WB Docs and links it to the incident record.
POST DownloadIncidentsListReport	Handler: DownloadIncidentsListReportQueryHandler → DownloadIncidentsListReportService. Generates incident list Excel/PDF report.
POST DownloadIncidentsDetailsReport	Handler: DownloadIncidentsDetailsReportQueryHandler → DownloadIncidentsDetailsReportService. Detailed incident report.

8.7 GbvDashboardController — /api/esrd/GbvDashboard

Dedicated GBV analytics dashboard.

POST GetGbvDashboardDetails (DashboardManager permission) and GetGbvDashboardDetailsForSpecialist (DashboardSpecialistView): GbvDashboardDetailsQueryHandler → GbvDashboardService.GetGbvDashboardDetails(). Body: GbvDashboardQuery with optional EsSpecialistUPI filter. Service aggregates GBV risk ratings across all pipeline and portfolio projects: count by rating category (Low/Medium/High/Medium FI/No Rating), breakdown by project phase, breakdown by management group, count of Final vs Pending ratings. POST DownloadGbvDashboardReport: Generates GBV analytics Excel using DownloadGbvDashboardReportsService.

8.8 EsInsightsController — /api/esrd/EsInsights

AI-powered document analysis using the Malena AI service.

Endpoint	Description
POST StartMalenaUpload	Handler: EsInsightsMalenaUploadCommandHandler → EsInsightService. Retrieves documents from WB Docs for the project.
POST GetMalenaResult	Handler: EsInsightsMalenaResultCommandHandler → EsInsightService. Polls Malena AI service for analysis results by job id.
GET GetEsInsightDocuments	Handler: GetEsInsightsDocumentsQueryHandler. Fetches list of E&S documents available for AI analysis from EsInsightsService.
GET GetPowerBiAccessToken	Handler: GetPowerBiAccessTokenQueryHandler. Calls PowerBiIntegrationRepository to obtain an Azure AD access token for Power BI.

GET GetPowerBiReportDetails	Handler: GetPowerBiReportQueryHandler. Returns Power BI workspace/report IDs for the ES Insights dashboard embed.
GET GetEsInsightsSentencewiseData	Handler: GetEsInsightsSentencewiseDataQueryHandler. Returns sentence-level analysis data for the ES Insights word cloud.
POST DownloadReport / DownloadReportWithModel	Download Report / Download Report With AI Model analyzed document with or without the AI model overlay.

9. BgJobController — Automated Background Processes

The Background Job controller exposes endpoints that are called by an external scheduler (Azure Logic Apps or equivalent) to run automated maintenance and notification tasks. These are NOT called by the UI. They are cron-like operations that keep system data current without user intervention. Route: /api/esrd/BgJob.

Endpoint	Handler	Service	Business Purpose
POST UpdateAmrCalendarYear	UpdateAmrCalendarYearCommandHandler	AmrHistoryBackgroundJobService	Periodically updates the AMR calendar year on EsHeader record.
POST CreateAmrHistory	CreateAmrHistoryCommandHandler	AmrHistoryBackgroundJobService	Creates new EsReviewRecord (AMR type) entries for all projects.
POST UpdateProjectPhase	UpdateProjectPhaseCommandHandler	ProjectPhaseBackgroundJobService	Checks MPM Integration Service for projects whose phase has changed.
POST UpdateKnowledgeGap	UpdateKnowledgeGapCommandHandler	KnowledgeGapBackgroundJobService	Re-synchronizes KnowledgeGap (IsKnowledgeGapActionPlan) with external data.
POST UpdateAmrDueDate	UpdateAmrDueDateCommandHandler	AmrDueDateBackgroundJobService	Calculates and updates AmrDueDate on EsHeader for all AMRs.
POST UpdateAmrDueDateAndKnowledgeGapDueDate	UpdateAmrDueDateAndKnowledgeGapDueDateCommandHandler	AmrDueDateBackgroundJobService	Calculates and updates both AMR due dates and knowledge gap due dates.
POST UpdateEsrrdDateAndRating	UpdateEsrrdDateAndRatingCommandHandler		Syncs ESRR(D) date and score from DdQuestionResponse to EsHeader.
POST UpdateEsrrmDateAndRating	UpdateEsrrmDateAndRatingCommandHandler		Syncs ESRR(M) date and score from MnQuestionResponse to EsHeader.
POST UpdateEsrrDateAndRatingToEsHeaderDateAndRatingToEsHeader	UpdateEsrrDateAndRatingToEsHeaderDateAndRatingToEsHeaderCommandHandler		Master ESRR(D/M) Job Service combines D and M rating updates.
POST UpdateEsapRequiredData	UpdateEsapRequiredDataCommandHandler	EsapRequiredBackgroundJobService	Synchronizes QuestionsResponse for the ESAP question (Questionnaire) to EsHeader.
GET GetSpecialistsProjects	GetSpecialistsProjectsQueryHandler	BGJobNotificationService	Fetches all pipeline + portfolio projects for a specific specialist.
GET GetManagerProjects	GetManagersProjectsQueryHandler	BGJobNotificationService	Fetches consolidated project view for a manager: pipeline, portfolio, and all.
GET GetAmrProjectsForSpecialist	GetAmrProjectsQueryHandler	BGJobNotificationService	Returns projects with upcoming or overdue AMRs for specialist.
GET GetDueDiligenceNotification	GetDueDiligenceNotificationQueryHandler	BGJobNotificationService	Projects with overdue or approaching DD deadlines for notification.
GET GetHighRiskProjectsNotification	GetHighRiskProjectsNotificationQueryHandler	BGJobNotificationService	High risk project details for manager notification emails.
GET GetMonitoringNotification	GetMonitoringNotificationQueryHandler	BGJobNotificationService	Portfolio projects with monitoring activities due soon.
GET GetEsapNotificationsData	GetEsapNotificationsDataQueryHandler	EsapNotificationService	ESAP actions that are overdue or due within a threshold — for manager notification.
GET GetEsrdWeeklySnapshot(Composite)		BGJobService	Master weekly snapshot endpoint. Based on isManager/isSpecialist.
GET GetUnassignedProjects	GetUnAssignedProjectsQueryHandler	BGJobNotificationService	Projects with no assigned E&S specialist — for manager notification.
GET GetGBVRiskRatingProjects	GetGBVRiskRatingProjectsQueryHandler	BGJobNotificationService	Projects needing GBV risk rating completion.

10. Additional Controllers — Quick Reference

10.1 DropDownValuesController — /api/esrd/DropDownValues

Single endpoint: GET GetDropDownValues. Permission: ReadAll. Handler: DropDownValuesQueryHandler → fetches all master data dropdowns needed by the UI in one call: regions, sectors, management groups, ESRR score options, GBV risk rating options, document types, initiative types, project categories, IFC track types, PS options, monitoring activity types. Returns DropDownValuesResponse. The data is sourced partly from the DB master tables and partly from the MasterDataCache. This is called once on app load.

10.2 EsrdMasterDataController — /api/esrd/EsrdMasterData

Admin-only (EditAll permission) master data management for GBV questionnaire tables. GET GetMasterDataInfo: returns all configurable master data tables (GBV questions, options, weights, country risk scores). GET GetGbvTableData (query: tableName): returns the contents of a specific GBV master table. GET DownloadGbvTableDataReport (query: tableName, isTemplate): downloads the table as Excel (data) or blank template. POST UploadGbvTableData [FromForm]: uploads a filled Excel template to bulk-update a GBV master data table. Handler: UploadGbvTableDataCommandHandler → MasterDataInfoQueryHandler → GbvTableDataQueryHandler.

10.3 ProvideClearanceController — /api/esrd/ProvideClearance

Manages the GBV clearance workflow where a GBV specialist provides clearance for a project. GET GetProvideClearanceProjects (ReadAll): returns projects pending GBV clearance via ProvideClearanceProjectsQueryHandler → ProvideClearanceProjectsService. POST ApproveProvideClearanceProject (UnAssignedRequestApprover permission only): ApproveProvideClearanceProjectCommandHandler — validated by ApproveProvideClearanceProjectCommandValidator. Service updates GbvHeader.GbvStatusCode to 'Cleared', sets ClearedOn=today, sends clearance confirmation email notification. POST DownloadProvideClearanceProjectsReport: generates Excel of clearance projects.

10.4 UnassignedRequestsController — /api/esrd/UnassignedRequests

Manages project assignment workflow for the GBV unassigned requests queue. GET GetUnassignedRequestProjects (ReadAll): UnassignedRequestQueryHandler → UnassignedRequestService. Returns projects submitted for assignment but not yet assigned to a GBV specialist. POST SaveUnassignedRequests (UnAssignedRequestApprover permission): SaveUnassignedRequestsCommandHandler → UnassignedRequestService. Assigns a GBV specialist to the project, sends assignment notification email.

10.5 OperationsController — /api/esrd/Operations (Internal)

An internal/test controller with [Authorize] (no policy check). Exposes GetPipelineProjects and GetPortfolioProjects using the same queries as ProjectsController but without ESRD policy enforcement. Intended for internal service-to-service calls (e.g., from background jobs or integration tests). Uses the same PipelineProjectsQueryHandler and PortfolioProjectsQueryHandler.

10.6 TestServiceController — /api/esrd/TestService (Dev Only)

Note: This controller is for initial setup testing ONLY. It does NOT appear in the Swagger docs for Production environment. Contains: TestApi (returns WbDocsI4KEY env var), TestAuthorization (JWT test), TestEmail (sends test email via IEmailNotification), TestWbDocsUpload (test file upload to WB Docs), TestAppInsights (logs a trace event).

11. Cross-Cutting Concerns

11.1 Authorization — PolicyAuthorize Attribute

Every endpoint (except BgJob and TestService) is protected by [RLAH.PolicyAuthorize(nameof(ApplicationCodes.ESRD), permissionCode)]. This custom attribute extends the standard ASP.NET Core [Authorize] and adds a MIGA-specific policy check. The permission codes used across the system are:

Permission Code	Used On	Description
EsrdPermissions.ReadAll	GET endpoints, report downloads	Any authenticated ESRD user can read data
EsrdPermissions.EditAll	POST save/delete endpoints	E&S Specialists and above can create/modify data
EsrdPermissions.DashboardManagerDashboard	Dashboard GetDashboardDetails, GetPipelineForm, GetPortfolioForm	Manager and Admins see all-member data
EsrdPermissions.DashboardSpecialistDashboard	Dashboard *ForSpecialist variants	Specialists see their own scoped analytics
EsrdPermissions.GbvDashboardManagerDashboard	GBV Dashboard manager endpoints	GBV managers see all GBV data
EsrdPermissions.GbvDashboardSpecialistDashboard	GBV Dashboard specialist endpoints	GBV specialists see own GBV data
EsrdPermissions.UnAssignedRequestsApprove	ApproveUnAssignedRequests, ApproveProvideClearance	Special approver role only
EsiaPermissions.InsightsDashboardView	ES Insights Power BI, sentence-wise data	ES Insights read access
[Authorize] only	EsInsights StartMalenaUpload/GetMalenaResults/Operations	Operations only, no ESRD policy check

11.2 FluentValidation — All Validators Reference

Validators are registered via services.AddValidatorsFromAssembly() and automatically invoked by MediatR's pipeline behavior before any handler executes. Here is a summary of all validators:

Validator Class	Command/Query	Key Rules
SaveDueDiligenceCommandValidator	SaveDueDiligenceCommand	For each QuestionDetail: If QuestionSection == ContextualRiskSection then...
SaveMonitoringActivityCommandValidator	SaveMonitoringActivityCommand	If AmrReviewFlag=true: AmrFromDate, AmrToDate, AmrDueDate, AmrReview...
SaveMonitoringActivityAndAmrHistoryCommandValidator	SaveMonitoringActivityAndAmrHistoryCommand	SaveMonitoringActivityCommand and SaveAmrHistoryCommand conditional date rules as SaveMonitoringActivityCommandValidator
SaveEsapActionCommandValidator	SaveEsapActionCommand	Action text required. DueDate required.
SaveGbvQuestionnairesCommandValidator	SaveGbvQuestionnairesCommand	All GBV questionnaire answers must be present for required questions.
SaveGbvActionPlanDetailCommandValidator	SaveGbvActionPlanDetailCommand	Action, DueDate required.
DeleteEsapActionCommandValidator	DeleteEsapActionCommand	esapActionIdList must not be empty.
DeleteGbvActionPlanDetailCommandValidator	DeleteGbvActionPlanDetailCommand	GbvActionPlanDetailId > 0 required.
SaveAdditionalInformationForPipelineCommandValidator	SaveAdditionalInformationForPipelineCommand	EsId required and > 0.
SaveAdditionalInformationForPortfolioCommandValidator	SaveAdditionalInformationForPortfolioCommand	EsId required and > 0.
SaveTeamInformationCommandValidator	SaveTeamInformationCommand	EsId and ProjectNumber required. EsSpecialistUpi must be > 0.
SaveOtherInitiativesCommandValidator	SaveOtherInitiativesCommand	Description required, Deadline required, InitiativeType required.
SaveNatureFinanceCommandValidator	SaveNatureFinanceCommand	EsId required. If IsNatureFinance=true: Justification required.

SaveIncidentCommandValidator	SaveIncidentCommand	EsId, IncidentDate, IncidentType, Description required.
UpdateDocumentNotesCommandValidator	UpdateDocumentNotesCommand	DocumentId required, Notes max length check.
GetEsapDetailsQueryValidator	GetEsapDetailsQuery	EsId must be > 0.
GetNatureFinanceQueryValidator	GetNatureFinanceQuery	EsId must be > 0.
ApproveProvideClearanceProjectCommandValidator	ApproveProvideClearanceProjectCommand	ProjectId and GbvId required. ClearanceDecision required.
AddOtherInitiativesTypeCommandValidator	AddOtherInitiativesTypeCommand	InitiativeType text required and max length.
UploadGbvTableDataCommandValidator	UploadGbvTableDataCommand	File required, Table name required, valid table name check.
DownloadGbvTableDataReportQueryValidator	DownloadGbvTableDataReportQuery	Table name required and must match valid table list.
GetManagersProjectsQueryValidator	GetManagersProjectsQuery	Upi must be > 0.
GetSpecialistsProjectsQueryValidator	GetSpecialistsProjectsQuery	Upi must be > 0.
UploadGbvActionPlanDetailsCommandValidator	UploadGbvActionPlanDetailsCommand	EsId > 0, File required.

11.3 AutoMapper Profiles — Data Transformation Layer

AutoMapper is configured to scan the entire assembly at startup. Every mapping is defined in a dedicated Profile class. The system has 30+ mapper profiles.

Mapper Profile	Maps From → To	Purpose
MonitoringMapperProfile	SaveMonitoringActivityCommand → EsMonitoring DdQuestionResponseSummary → MnQuestionResponseSummary EsHeader → MonitoringQuestionnaire EsMonitoring → MonitoringActivityInformation	Creating monitoring entity transformations. The DD→MN mapping seeds the database.
DueDiligenceMapperProfile	SaveDueDiligenceCommand.GeneralInfo → DdQuestionResponseSummary DdQuestionsResponse → QuestionDetail EsHeader → DueDiligenceQuestionnaire	Due diligence entity transformations.
EsapActionMapperProfile	SaveEsapActionCommand → EsapAction EsapAction → EsapActionInformation/EsapActionInformationDto	ESAP action CRUD transformations.
PortfolioProjectsTabMapperProfile	PortfolioProjectsEntityModel → PortfolioProjectInformation	Maps raw DB portfolio data to the response model.
ProjectAndApplicationDetailsResponseMapperProfile	ProjectAndApplicationDetails → PortfolioProjectInformation	Maps integration service data into portfolio project response.
EsHeaderPipelineProjectMapperProfile	EsHeader → PipelineProjectInformation PipelineProjectsEntityModel → PipelineProjectInformation	Pipeline project mapping from DB entities.
PipelineDetailsToPipelineProjectMapperProfile	ProjectAndApplicationDetails → PipelineProjectInformation	Maps integration service data to pipeline response.
GbvDetailMapperProfile	GbvHeader → GbvDetailsResponse SaveGbvQuestionnairesCommand → GbvQuestionsResponse	GBV questionnaire and response mappings.
GbvActionPlanDetailMapperProfile	SaveGbvActionPlanDetailCommand → GbvActionPlanDetail	GBV action plan CRUD.
HighRiskMapperProfile	SaveHighRiskDetailsCommand → HighRiskProjectInfo	High risk data transformation.
NatureFinanceMapperProfile	SaveNatureFinanceCommand → NatureFinanceEntity	Nature finance save mapping.
OtherInitiativesMapperProfile	SaveOtherInitiativesCommand → OtherInitiativesEntity	Other initiatives CRUD mapping.
DocumentMapperProfile	SaveDocumentCommand → EsDocument EsDocument → EsDocuments DTO	Document entity transformations.
IncidentMapperProfile	SaveIncidentCommand → Incident entity	Incident save mapping.

AuditSummaryMapperProfile	AuditLog → AuditSummaryEntry DTO	Audit log display transformation.
DashboardMapperProfile	EsHeader → DashboardProjectsSummary Multiple → DashboardDetails sections	Dashboard aggregation mappings.
AmrHistoryMapperProfile	AmrHistory → AmrHistoryDetails DTO	AMR history display mapping.
SaveAdditionalInformationForPipelineSaveAdditionalInformationForPipelineCommand	SaveAdditionalInformationForPipelineCommand → PipelineEntity	Pipeline additional info save.
SaveAdditionalInformationForPortfolioSaveAdditionalInformationForPortfolioCommand	SaveAdditionalInformationForPortfolioCommand → PortfolioEntity	Portfolio additional info save.
TeamInformationMapperProfile	SaveTeamInformationCommand → EsTeam	Team assignment mapping.
ShelvedProjectsMapperProfile	ShelvedProjectsEntityModel + Integration data	Shelved Projects response mapping.
ReportsMapperProfile	Various DB entities → Report DTOs	Report data transformation mappings.
ESInsightMapperProfile	ESInsightsDocument → ESInsightsDocument	ES Insights document list mapping.

11.4 Domain Event Bus — Async Integration

The system uses Azure Service Bus for asynchronous domain event communication. Events are defined in the EventBus/DomainEvents folder and handlers in EventBus/DomainEventHandlers.

Domain Event	Event Handler	Trigger	Action
ProjectPhaseUpdateEvent	ProjectPhaseUpdateEventHandler	Background job UpdateProjectPhase	UpdateProjectPhase endpoint. Project phase changes.
ESInsightsMalenaPostEvent	ESInsightsMalenaPostEventHandler	StartMalenaUpload endpoint	Async handler that triggers the Malena AI document upload.
ESInsightsMalenaGetEvent	ESInsightsMalenaGetEventHandler	GetMalenaResult endpoint	Async handler that polls the Malena AI service for analysis.
ClimateTeamInfoUpdatedInClimateTeam	ClimateTeamInfoUpdatedInClimateTeamEventHandler	ClimateTeamInfoUpdatedInClimateTeam endpoint	Cross-portal team synchronization (currently disabled).
TeamAssignmentApprovedEvent	no explicit handler — published via SaveTeamEvent	SaveTeamEvent endpoint	Published when a team assignment is created/updated to n

12. Database, Configuration & Infrastructure

12.1 PostgreSQL Database Connection

The connection string is fetched from Azure Key Vault at startup using the Secrets class with key 'DbConnStrESRD'. The EF Core DbContext (ESReviewDocumentationDbContext) uses the Npgsql provider. AppContext.SetSwitch('Npgsql.EnableLegacyTimestampBehavior', true) is called to handle DateTime mapping compatibility. Schema migrations are stored in a custom table: schema=APPLICATION_SCHEMA, table=__EFMigrationsHistory.

12.2 Environment Variables (Key Configuration)

```
MPMIntegrationServiceUrl  - Base URL of the MPM Integration Service (MIGA portal integration)
NotificationServiceUrl    - Base URL of the WBG email notification service
MalenaServiceUrl          - Base URL of the Malena AI document analysis service
WbDocsFileUrl             - WB Docs file URL template (format string with SiteKey)
WbDocsSiteKey             - Current WB Docs site key
WbDocsSiteKey_Old         - Legacy WB Docs site key (for older documents)
WbDocsI4KEY               - WB Docs API authentication key
Environment               - Environment name: Dev/UAT/Production (controls Swagger visibility)

// Azure Key Vault secrets (fetched at startup via Secrets class):
DbConnStrESRD             - PostgreSQL connection string
app-insights-connectionstring - Azure Application Insights connection string
MalenaSubscriptionKey     - Subscription key for Malena AI service headers
```

12.3 Swagger UI Configuration

Swagger is ONLY enabled for non-Production environments. The Swagger UI is accessible at: /api/esrd/swagger. It is configured with Bearer JWT authentication — users must enter their JWT token in the Authorize dialog to test endpoints. CustomSchemas uses full type names (including nested class separators converted from '+' to '.') to avoid schema ID conflicts across the many similarly-named request/response types.

12.4 Master Data Cache

At startup, masterDataRepository.InitializeMasterDataCache() is called in a background Task.Run(). This populates an in-memory cache with: EsMasterData (project phases, ESRR scores, IFC tracks, etc.), country codes, region codes, sector codes, management group codes. All CommonService methods that return lookup values read from this cache (GetEsMasterData, GetEsrrScoreText). This avoids repeated DB queries for static reference data on every API call.

12.5 WB Docs Integration

services.InitializeWbDocs() registers the IWbDocs interface from the MIGAPortal.RL.Utilities library. Used in DocumentService, IncidentService, and EsInsightService to upload files to the World Bank's WB Docs document management system. Files are uploaded with metadata (FileName, FileExtension, FileContent). The WbDocsFileUrl template constructs browseable document URLs for the UI.

12.6 Response Envelope — Standard API Contract

```
// Every API response uses this envelope (PropertyNameingPolicy=null → PascalCase):
{
  "STATUS": "Success",      // 'Success' or 'Error'
  "STATUS_CODE": 0,         // 0 = Success, non-zero = error code
  "NOTIFICATIONS": [        // Empty array on success; error details on failure
    {
      "Code": "500",
      "Message": "EsId not found",
      "NotificationType": "Error"
    }
  ],
  "Result": { ... }         // Actual data payload – type varies per endpoint
}

// File download endpoints return FileStreamResult instead of JSON:
// Response headers: Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
// Content-Disposition: attachment; filename="ReportName.xlsx"
```